

自然语言处理基础 课程笔记

25-26 学年第 2 学期

AutoPku

2026-05-16

Contents

自然语言处理基础—课程笔记	2
0. 课程脉络	2
1. 课程概览	2
2. 中英文术语对照	2
3. 期末复习指引	3
Lecture 10 (II) —IR Embedding (向量检索)	4
1. 为什么需要 Embedding Learning	4
2. Formulation: Representation-centric 视角	5
3. 双塔架构 (Dual Encoders)	5
4. 近似最近邻搜索 (ANNS)	6
5. 训练: Contrastive Learning / Learning to Rank	7
6. 难负例 (Hard Negatives): 本讲的真正主题	8
7. 总结: Embedding Learning 的设计哲学	10
8. 章节内速查表	10
9. 整体关系图	11
10. 期末复习要点	11
11. 参考文献 (课件 Readings)	12
Lecture 11 —预训练模型 (Pretrained Language Models)	13
1. Motivation: 为什么要预训练	13
2. ELMo: 上下文相关表示的起点	14
3. Pre-training & Fine-tuning 范式	15
4. Transformer 架构核心回顾	15
5. BERT: 双向编码器表示	16
6. RoBERTa: 把 BERT 训得更彻底	19
7. Encoder-only 的局限	19
8. T5: Text-to-Text Transfer Transformer	19
9. GPT 系列: 自回归解码器	20
10. 模型家族 mermaid 图	22
11. 下游适配范式	22
12. 关键模型对比表	23
13. 期末复习要点	23

自然语言处理基础—课程笔记

[TIP] 课程定位

本课程是 PKU 计算机/信科类的 NLP 入门核心课。由经典统计 NLP（语言模型、词义消歧、HMM、句法分析、统计机器翻译）一路推进到向量检索与预训练大语言模型范式。期末考会涵盖语言建模、对数线性模型、HMM、CRF、PCFG/CKY、统计 MT、向量检索与预训练。

0. 课程脉络



1. 课程概览

周	讲次	主题	关键概念
1	Lecture 1	导论	NLP 任务谱系、歧义、评测
2	Lecture 2	词义消歧 (WSD)	Lesk、有监督特征工程
3	Lecture 3	分类 / 对数线性	NB、感知机、Logistic、特征模板
4	Lecture 4	语言模型	n-gram、平滑、神经 LM、困惑度
5	Lecture 5	分布式语义	共现矩阵、SVD、word2vec、GloVe
6	Lecture 6	序列标注	HMM、Viterbi、forward-backward、CRF、seq2seq
7	Lecture 7	短语句法	CFG、CKY、PCFG
8	Lecture 8	依存句法	shift-reduce、Eisner、UAS/LAS
9	Lecture 9	统计机器翻译	IBM Model 1-2、phrase-based、BLEU
10 (I)	Lecture 10-IR	信息检索 (稀疏)	Boolean、TF-IDF、BM25
10 (II)	Lecture 10-Embed	IR Embedding (稠密)	双塔、InfoNCE、in-batch / hard negatives、ANNS
11	Lecture 11	预训练语言模型	Transformer、BERT、GPT、T5/BART、ELECTRA、ELMo、Scaling、tokenization

既有 .tex 笔记 (fnlp_notes.tex, fnlp_midterm_review.tex) 覆盖 Lec 1-9 期中知识点，本目录以 lec*.md 形式增量记录 Lec 10 (II) 起的新内容。

2. 中英文术语对照

中文	English	备注
词义消歧	Word Sense Disambiguation (WSD)	Lec 2
对数线性模型	Log-linear model	MaxEnt / 多类 Logistic
语言模型	Language Model (LM)	n-gram, Neural LM, MLM, CLM
困惑度	Perplexity (PPL)	$\exp(-\frac{1}{N} \sum \log p(x))$
词向量	Word Embedding	static (word2vec/GloVe) vs contextual (ELMo/BERT)
共现矩阵	Co-occurrence matrix	PPMI, SVD
隐马尔可夫模型	Hidden Markov Model (HMM)	概率图、生成式
条件随机场	Conditional Random Field (CRF)	判别式、全局归一
维特比算法	Viterbi algorithm	序列解码
前向-后向	Forward-backward	HMM 边缘 / 期望
上下文无关文法	Context-Free Grammar (CFG)	
CKY 算法	CKY parsing	Bottom-up chart
概率 CFG	Probabilistic CFG (PCFG)	
依存句法	Dependency parsing	转移 / 图模型
BLEU	BLEU score	MT 自动评测
信息检索	Information Retrieval (IR)	Lec 10
稀疏检索	Sparse retrieval	BM25 等
稠密检索	Dense retrieval	DPR、ANCE 等
双塔	Dual-encoder / bi-encoder	query / doc 独立编码
对比学习	Contrastive learning	InfoNCE、SimCSE
近似最近邻	Approximate Nearest Neighbor (ANN)	FAISS、HNSW、IVF-PQ
检索增强生成	Retrieval-Augmented Generation (RAG)	
自注意力	Self-attention	$\text{softmax}(QK^T/\sqrt{d})V$
多头注意力	Multi-head attention	
掩码语言模型	Masked LM (MLM)	BERT
因果语言模型	Causal LM (CLM)	GPT
替换词检测	Replaced Token Detection	ELECTRA
去噪 / 编码-解码	Denoising encoder-decoder	BART, T5
字节对编码	Byte-Pair Encoding (BPE)	tokenization
WordPiece / SentencePiece	WordPiece / SentencePiece	BERT / T5
提示学习	Prompt-based learning	zero/few-shot
指令微调	Instruction tuning	FLAN, T0
上下文学习	In-context Learning (ICL)	GPT-3
缩放律	Scaling Laws	Kaplan / Chinchilla

3. 期末复习指引

- **统计 NLP 基础 (Lec 1-6)** : 语言模型概率、平滑、HMM/CRF 解码、对数线性梯度。已有 .tex 期中复习。
- **句法 + 统计 MT (Lec 7-9)** : CKY / PCFG、IBM Model 1 期望最大化、BLEU 评测。
- **现代范式 (Lec 10-11)** : 必须掌握 dense retrieval 对比训练损失、Transformer self-attention 公式、各预训练目标 (MLM/CLM/ETD/Denoising) 的数学描述。

[WARN] 复习建议

1. 把 Lec 5 (word embedding) 与 Lec 10(II) (sentence/passage embedding) 串起来: 从静态词向量到 contrastive learning 的训练目标演进。2. Lec 11 中关注 BERT 与 GPT 的目标函数和适用任务的对比; Scaling Law、Chinchilla 结论可能以选择题/简答出现。3. ANNS 三类方法 (partition / hash / graph) 和代表算法 (IVF, LSH, HNSW) 需要会简要描述。

Lecture 10 (II) —IR Embedding（向量检索）

[TIP] 学习指南

本节核心：把文本（query 与 document）编码到同一个向量空间，用向量相似度近似“语义相关性”。重点理清三件事：1. **双塔 (dual-encoder) 架构**——离线编码、在线只做向量运算；2. **对比训练 (contrastive learning) + 难负例采样**——为什么随机负例不够、Sparse Retrieval Negatives 与 Self Negatives (ANCE) 的对照；3. **近似最近邻 (ANNS)**——partition / hash / graph 三类方法如何把检索成本降到 sub-linear。
前置：- lec5_dist (distributional semantics、cosine similarity、word/sentence embedding) - lec10_ir (IR 任务定义、TF-IDF、BM25、Boolean / Vector Space / Probabilistic / Neural Model；DPR 的 in-batch negatives 是本讲的起点)
复习关注点：dual-encoder 的相关性函数、对比损失形式、为什么 corpus 99.9999% trivially irrelevant、ANCE 的“周期性刷新索引”思想、ANNS 三类方法各自代表算法。

1. 为什么需要 Embedding Learning

上一节 (lec10_ir) 走完了 Boolean → Vector Space → BM25 → Neural Model 的脉络，DPR (Dense Passage Retrieval) 让“用稠密向量做召回”成为可行方案。本讲把镜头拉近，专门讨论 **embedding learning** 这一独立的研究主题。

Embedding Learning：把文本（也可推广到用户、商品、图像等）编码到统一向量空间 $\mathbb{R}^h$ ，让“语义关系”对应“向量几何关系” (cosine / 内积)。

它的下游用途横跨 NLP 与推荐系统：

场景	由 embedding 捕获的语义
搜索 / RAG	query-document 相关性
推荐系统	用户兴趣与商品语义
聚类 / 去重	数据间相似性
多模态	跨模态对齐

1.1 三大动机：Efficiency First

课件强调，embedding 之所以被大规模工业系统采用，根本原因是 **efficiency**：

1. **Offline pre-computed embedding**：document 侧的向量可以离线一次性算好，写进磁盘 / 向量库。在线只需要算 query 向量。
2. **Quick similarity calculations**：检索归结为一次向量内积或 cosine，远比走神经网络对每个 (q, d) 配对打分便宜。
3. **Approximate Nearest Neighbor Search (ANNS)**：成熟的近似 KNN 算法把检索复杂度降到 sub-linear，可与倒排索引 (inverted index) 相提并论。

[NOTE] 工业流水线视角

Embedding-based 召回是当今搜索 / 推荐流水线的第一阶段 (“first stage”)。后面通常还有 re-ranking (精排), 用 cross-encoder 或更重的模型在召回的少量候选上再打分。Embedding 的目标是 **recall**, 把相关候选放进来; 精排负责 **precision**。

2. Formulation: Representation-centric 视角

回顾 IR 任务定义: 要计算 query q 与 document d 的相关性 $f(q, d)$ 。Embedding 方法的口号是:

Turn everything into vectors. Only simple vector operations afterwards.

也就是说, 我们只关心两个 **encoder**: query encoder E_q 和 document encoder E_d 。打分函数限制为一种简单的向量运算:

$$f(q, d) = E_q(q)^\top E_d(d). \quad (1)$$

这种把 f 限制成“向量内积”的设计被称为 **representation-centric**, 它牺牲了表达力 (query 与 document 没有任何 token-level 交互), 换来三件事:

- document 向量可离线算;
- query 上来只前向一次;
- 整个 corpus 可建 ANN 索引。

[WARN] 与 interaction-centric 的取舍

Interaction-centric 方法 (如 cross-encoder: 把 $[q; d]$ 一起塞进 BERT 做分类) 允许 token-level 注意力, 效果更好但**无法预先索引**——每来一个 q , 都要对 corpus 中**每个** d 重新跑一次 BERT。本讲完全选 representation-centric 一侧。

3. 双塔架构 (Dual Encoders)

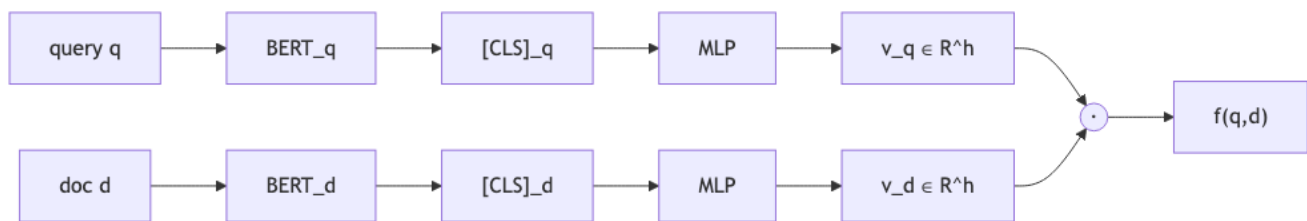
3.1 BERT 版本的实例

课件以 BERT 为骨干给出最简形式:

$$f(q, d) = \text{BERT}(q) \times \text{BERT}(d) = \text{MLP}([\vec{\text{CLS}}]_q) \times \text{MLP}([\vec{\text{CLS}}]_d). \quad (2)$$

实现要点:

1. 两个**独立** BERT (也可以共享参数 $\rightarrow$ “siamese”; DPR 用的是**两个独立**的 encoder, 因为 query 与 passage 分布不同)。
2. 取 [CLS] token 的最后一层 hidden state 作为整段文本的表示。
3. 后接一个 MLP 投影到检索空间 (可选, 但常用于降维和归一化)。
4. 相关性函数定为 **dot product** (或先 ℓ_2 -norm 再 dot product, 即 cosine)。



3.2 Inference 流程

```
# 离线: 把 corpus 全部编码并建索引
for d in corpus:
    v_d = MLP(BERT_d(d).cls)      # 一次性
    index.add(v_d, doc_id=d.id)
index.build_ann()                # FAISS / HNSW 等

# 在线: 每来一个 query
v_q = MLP(BERT_q(q).cls)
top_k = index.search(v_q, k=K)    # 近似 KNN
```

3.3 为什么 inference 仍然“很贵”?

课件特意问了一句 “Inference can be very expensive, why?”. 原因有两层:

1. **语料规模**: corpus 可能是百万 / 十亿 / 万亿级, 朴素地与每个 v_d 算内积是 $O(|D| \cdot h)$ 。
2. **维度诅咒**: h 通常 256—768, 且要保证检索精度, **精确 KNN** 不实用。

这就引出下一节的 ANNS。

4. 近似最近邻搜索 (ANNS)

目标: 以“略微牺牲 KNN 精度”为代价, 把检索做到 **sub-linear** 时间。

ANNS 把 K -nearest 由精确变近似, 三大类方法:

4.1 Partition-based

把向量空间划分成若干区域, 查询时只搜部分区域。

- **代表算法**: hierarchical K-means trees (先做 K-means 把空间分成 K 个 cluster, 每个 cluster 再递归 K-means)。
- **查询**: 先决定 v_q 落入哪个/哪几个 cluster, 只在这些 cluster 的成员里做精确 KNN。

4.2 Hash-based

用哈希函数把向量映射成离散 hash code, 只检索“同 hash 桶”或“近邻 hash 桶”中的点。

- **代表算法**: **Locality Sensitive Hashing (LSH)** ——满足“距离近的点 hash collision 概率高”。
- 多组 hash 函数 ensemble, 召回率随 hash table 数量上升。

4.3 Graph-based

把 corpus 中每个点与其相似邻居用边连起来，查询时贪心地在图上走。

- 代表算法：K-nearest neighborhood graph（每个点与最近的 K 个点连边）、HNSW（层次化跳表式 KNN graph）。
- 查询：从入口点出发，每一步跳到当前节点邻居中与 v_q 最近的一个，直到收敛。

4.4 三类方法速查

类别	思想	代表	优点	弱点
Partition	空间划分	hierarchical K-means	直观；离线建索引快	边界点易丢；不易增量
Hash	离散投影	LSH	理论保证；易并行	高维下 collision 难控
Graph	图上贪心	KNN graph / HNSW	召回精度高；查询快	内存大；建图慢

[TIP] 工业实践

FAISS / ScaNN / HNSWlib 都是常见库。课件结论：ANNS 的成本可以与 inverted index（稀疏检索）持平——这是 dense retrieval 能进入生产管线的关键。

5. 训练：Contrastive Learning / Learning to Rank

5.1 一般形式

训练 dual-encoder 等价于解

$$\arg \min_{\theta} \sum_q \sum_{d^+} \sum_{d^-} \ell(f(q, d^+), f(q, d^-)). \quad (3)$$

其中：

- (q, d^+) 是给定的正例对（“q 与 d 相关”，常来自标注数据集；
- d^- 是负例（“q 与 d 不相关”，需要自己采样；
- ℓ 是 ranking loss，要求 $f(q, d^+) > f(q, d^-)$ 。

5.2 InfoNCE：把负例打包成 softmax

工业最常用的对比损失是 InfoNCE（softmax-form contrastive loss）。给定 query q 、正例 d^+ 和 N 个负例 $\{d_i^-\}_{i=1}^N$ ：

$$\mathcal{L}_{\text{InfoNCE}}(q, d^+, \{d_i^-\}) = -\log \frac{\exp(f(q, d^+)/\tau)}{\exp(f(q, d^+)/\tau) + \sum_{i=1}^N \exp(f(q, d_i^-)/\tau)}. \quad (4)$$

- τ 是 temperature；
- 直观上把 ranking 问题转成“在 $N + 1$ 个候选里挑出 d^+ 的 softmax 分类”。

[NOTE] DPR 的 in-batch negatives (lec10_ir 已讲)

设一个 mini-batch 含 n 对 (q_i, d_i^+) 。把 query embedding 拼成 $Q \in \mathbb{R}^{d \times n}$ 、passage embedding 拼成 $P \in \mathbb{R}^{d \times n}$ ，相似度矩阵

$$S = Q^\top P \in \mathbb{R}^{n \times n}.$$

对第 i 行做 softmax，目标是让对角元 S_{ii} 最大——其余 $n - 1$ 个 passage 自动成为 q_i 的负例。一次前向算 n^2 个 pair，效率极高。

6. 难负例 (Hard Negatives): 本讲的真正主题

6.1 现象：随机负例为什么不够

课件给出关键实验观察：在 MSMARCO 上用**随机负例**训练 dense retriever，loss 早早收敛——模型对随机负例已经几乎完美，但检索效果远未饱和。

根源：retrieval 的独特属性。

- Corpus 巨大：百万至万亿级别；
- 99.9999% 的样本 trivially irrelevant——从语料里随机抽一个 doc，几乎肯定与 query 八竿子打不着；
- 真正考验模型的是少量 hard negatives：那些“看起来很像但不答 query”的文档。

[WARN] 关键直觉

Retrieval 的本质不是“判断相关 / 不相关”——而是“在数百万个看似相关的候选里挑出真正相关的那几个”。所以训练信号必须由 hard negatives 提供。

6.2 难负例采样三策略

6.2.1 Sparse Retrieval Negatives

用一个已有的**稀疏检索系统** (BM25 / inverted index) 跑一遍 q ，把它返回的 top-K 中**不含正确答案**的文档当负例。

代表工作：

- Bing Vector Search [Waldburger 2019]: 负例直接来自工业级 inverted index；
- DPR [Karpukhin et al. 2020]: 负例 = BM25 top-K 中不含答案串的 passage；
- Facebook Embedding Search [Huang et al. 2020]: 离线从生产系统中挖 hard negatives。

维度	评价
Pros	可在已有系统上 bootstrap；负例语义有意义
Cons	对 dense retriever 仍然偏 trivial；out-of-domain 泛化弱

核心局限：稀疏检索难以判断的负例（词汇重叠高但语义不同），对稠密检索来说不一定难——反之亦然。两个系统的“难”分布不一致，导致 negative 信号不够锐利。

6.2.2 Self Negatives (ANCE 系列)

Approximate nearest neighbor Negative Contrastive Estimation (ANCE) [Xiong et al. 2020, arXiv:2007.00808] 提出：

让 dense retriever 自己找自己的难负例。

流程：

1. **Warm up**: 先用 sparse retrieval negatives (如 BM25 hard negatives) 训练几个 epoch, 得到一个初步可用的 retriever;
2. **Async refresh**: 定期用当前 retriever 对整个 corpus 重建 ANN 索引;
3. **Self mining**: 对每个 q , 从该索引返回的 top-K 中不含正例的文档作为新一轮 hard negatives;
4. **Continue training**: 继续训 → 回到第 2 步。

```
# ANCE 训练框架 (伪代码)
retriever = warmup_with_bm25_negatives()
for epoch in range(E):
    # 周期性刷新索引
    if epoch % refresh_interval == 0:
        index = build_ann_index(retriever, corpus)

    for batch in data:
        q, d_pos = batch
        # 用当前 retriever 自己挖 hard negatives
        candidates = index.search(retriever.encode(q), top_k=K)
        d_neg = [c for c in candidates if c != d_pos][:n_neg]
        loss = infoNCE(q, d_pos, d_neg)
        loss.backward(); optimizer.step()
```

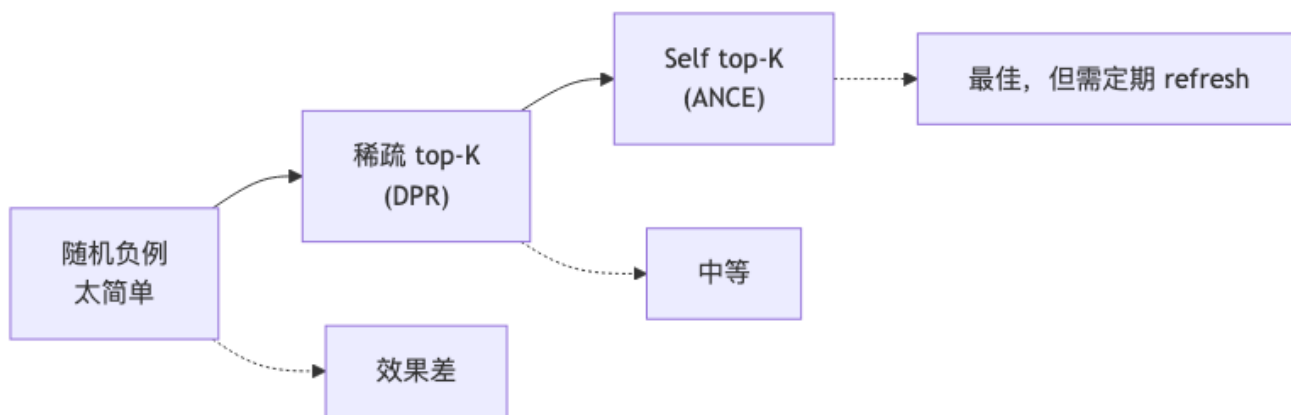
维度	评价
Pros	训练与测试分布对齐; in-domain & out-of-domain 表现都强
Cons	每次刷新索引代价大; 刷新前后训练不稳定 (gradient distribution shift)

[TIP] 对齐 train/test

ANCE 的核心洞察：测试时模型面对的是它自己排出来的 top-K；训练时也应该用它自己排出来的 top-K 作为难负例。如此 train/test gap 才最小。

6.2.3 三种负例策略对照

策略	负例来源	Train/Test 分布	工程代价	典型工作
Random Sparse Retrieval	语料随机抽 BM25 / inverted index top-K	偏差大 部分对齐 (稀疏侧)	极低 低 (已有系统)	早期 baseline DPR, Bing, FB
Self (ANCE)	dense retriever 自 身 top-K	完全对齐	高 (周期刷新)	ANCE



7. 总结: Embedding Learning 的设计哲学

课件结尾的 take-aways:

1. 把所有数据点映射到 **embedding 空间**——文本、图像、用户、商品都同框。
2. 用 **embedding 相似度**近似语义关系——cosine / dot product 就够用。
3. 借助 **ANN** 实现高效查询——以略损精度换 sub-linear 检索。
4. 训练需要特别处理:
 - 损失函数: **contrastive loss** (InfoNCE);
 - 负例选择: **sophisticated negative mining** (hard / self / cross-batch), 是真正的研究焦点。
5. **trade-off 思维**: representation-centric 牺牲表达力换效率, dense retrieval 牺牲精确 KNN 换 sub-linear, hard negative mining 牺牲训练稳定换分布对齐。

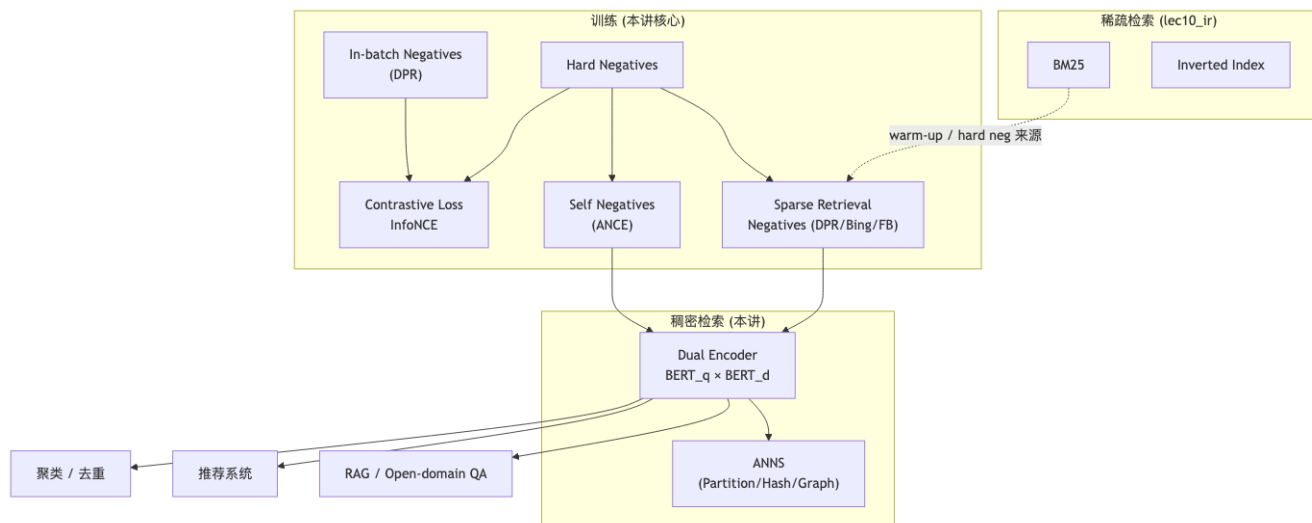
[NOTE] 课件留的 to-do

“To do: survey popular text embedding models, for processing Chinese, and many other languages.”
——课程鼓励大家自行调研 BGE、E5、Contriever、GTE、M3E 等多语 / 中文 embedding 模型, 看它们的训练数据、负采样、模型尺寸如何选择。

8. 章节内速查表

概念	定义 / 关键点
Dual encoder	两个独立编码器, $f(q, d) = E_q(q)^\top E_d(d)$
Representation-centric	只算向量; 可离线索引; 牺牲交互信息
ANNS	近似最近邻, sub-linear 查询
Partition method	空间划分; hierarchical K-means
Hash method	LSH; 同 hash 桶 $\approx$ 相邻
Graph method	KNN graph / HNSW; 图上贪心
In-batch negatives	mini-batch 内其他正例当负例; 相似度矩阵 $S = Q^\top P$
Hard negative	看似相关但实际不相关; 训练 retriever 的关键信号
Sparse retrieval negatives	取 BM25 top-K 中非正例 (DPR)
Self negatives (ANCE)	用 retriever 自己 top-K, 周期刷新
InfoNCE	softmax 形式的对比损失, 温度 τ
MSMARCO	大规模 passage retrieval 数据集; 常用 benchmark

9. 整体关系图



10. 期末复习要点

1. 公式默写

- 双塔相关性: $f(q, d) = \text{MLP}([\text{CLS}]_q)^\top \text{MLP}([\text{CLS}]_d)$;
- 对比训练目标: $\arg \min_{\theta} \sum_{q, d^+, d^-} \ell(f(q, d^+), f(q, d^-))$;
- InfoNCE: 式 (4);
- In-batch negatives 相似度矩阵: $S = Q^\top P \in \mathbb{R}^{n \times n}$, 对角元最大化。

2. 概念辨析

- Representation-centric vs interaction-centric;
- Bi-encoder (本讲) vs cross-encoder (精排, 未本讲展开);
- 三种 ANN 方法 (partition / hash / graph) ——各自代表算法;
- 三种负例策略——pros & cons。

3. 可能考查的开放题

- “随机负例为什么不够?” → 99.9999% trivially irrelevant, retrieval 难度在 hard negatives;
- “BM25 hard negative 与 ANCE self negative 的差异?” → 分布对齐性、refresh 代价、generalization;
- “为什么 dual encoder 比 cross encoder 适合做召回?” → 可离线索引、可上 ANN, 工程效率。

4. 与其他章节的连接

- lec5_dist: cosine 相似度、distributional hypothesis 是 embedding 的语义基础;
- lec10_ir: BM25 / 倒排索引 / DPR / in-batch negatives ——本讲是其 embedding 视角的延伸;
- 后续 RAG / Open-domain QA: dense retriever 是 retriever-reader 流水线的左半边。

11. 参考文献（课件 Readings）

1. Karpukhin et al. Dense Passage Retrieval for Open-Domain Question Answering, EMNLP 2020. — DPR、in-batch negatives。
2. Huang et al. Embedding-based Retrieval in Facebook Search, KDD 2020, pp. 2553–2561. —工业级 hard negative mining。
3. Xiong et al. Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval (ANCE), arXiv:2007.00808. —Self negatives。
4. Waldburger 2019 (Bing Vector Search 报道) ——sparse retrieval negatives 工业案例。

Lecture 11 —预训练模型 (Pretrained Language Models)

[TIP] 学习指南

本节核心：从「为每个任务单独训模型」过渡到「先用海量无标注语料预训练，再用任务数据微调」。围绕三类架构 (encoder-only / decoder-only / encoder-decoder) 讲清楚 ELMo → BERT → RoBERTa → T5 → GPT-1/2/3 这条主线，并理解 MLM、Causal LM、Span Corruption 三种自监督目标在数学上的区别。**前置知识**：- lec4_lm —— n -gram 语言模型、困惑度 (PPL) - lec4_nlm —— FFNN-LM、RNN/LSTM 语言模型 - lec5_dist —— word2vec、分布式表示 - 注意力与 Transformer (lec9 / lec10) —— attention、multi-head、位置编码 **重点掌握**：1. MLM 与 Causal LM 的目标函数形式及梯度来源差异 2. BERT 的 15% 掩码细则 (80/10/10 比例) 与 NSP 的设计动机 3. ELMo 是「特征注入」、BERT/GPT 是「权重复用」这一范式区分 4. 三类架构与对应自监督目标的对应关系 5. GPT-1 → GPT-2 → GPT-3 的参数 / 数据 / 算力 scaling 轨迹及范式从 fine-tuning 转向 prompting 的动因

1. Motivation：为什么要预训练

1.1 静态词向量的局限

word2vec、GloVe 等方法对每个词只学习一个向量，无法处理一词多义。考虑英文 play 的四种用法（区域 / 行动 / 比赛 / 戏剧），静态向量无法区分：

句子片段	play 含义
Chico Ruiz made a spectacular play on Alusik's grounder	比赛动作
The new-look play area is due to be completed	玩耍区域
representatives who play to the party base	迎合行为
Mike was praised for his all-round excellent play last night	比赛表现
John signed to do a Broadway play for Garson	戏剧

只有第 (C) 句的 play 与原句语义最接近，但 word2vec 给出的全是同一个向量。上下文相关表示 (contextual representations) 应运而生。

1.2 表示学习 + 数据效率

预训练-微调 (Pre-training & Fine-tuning) 的核心论证：

在大规模任务 X 上预训练得到的模型，可以为相关下游任务 Y 提供可泛化的表示能力；找到一个对广泛 Y 都有用的 X，下游只需少量数据 + 少量任务参数即可达到优秀效果。

这意味着：– 大规模无标注语料一次性吃下，得到通用表示 – 下游任务训练数据再少也能用上海量预训练知识 – 模型权重几乎全部复用，只在顶层添加任务头

1.3 历史脉络

课程给出的演进路线：

VSM → PLSA (2003) → LDA (2003) → Neural LM (2003) → word2vec (2013/14) → RNN-LM / LSTM-LM (2015) → **Transformer (2017)** → **ELMo (2018-02)** → **GPT (2018-06)** → **BERT (2018-10)** → GPT-2 (2019) → XLNet (2019) → RoBERTa (2019) → T5 (2019) → GPT-3 (2020) →

2. ELMo：上下文相关表示的起点

2.1 模型结构

ELMo (Deep Contextualized Word Representations, Peters et al., 2018-02) 训练两条独立的多层 LSTM 语言模型：

- 前向 LM: $p(t_k | t_1, \dots, t_{k-1})$
- 后向 LM: $p(t_k | t_{k+1}, \dots, t_N)$

对每个 token k 与每个 LSTM 层 j ，分别得到隐状态 $\vec{h}_{k,j}$ 与 $\overleftarrow{h}_{k,j}$ ，拼接为 $h_{k,j} = [\vec{h}_{k,j}; \overleftarrow{h}_{k,j}]$ 。

最终 ELMo 表示是各层的任务相关加权平均：

$$\text{ELMo}_k^{\text{task}} = \gamma^{\text{task}} \sum_{j=0}^L s_j^{\text{task}} h_{k,j}$$

其中 s_j^{task} 是 softmax 归一化的层权重， γ^{task} 是任务相关的缩放因子。

[NOTE] 为什么要前向 + 后向？

单向 LM 只看到一侧上下文，无法消歧出现在句子开头的歧义词。前向 + 后向分别建模能让 token 获得双侧信息（但仍是两个独立模型，不是真正双向）。

为什么不只取最顶层？ELMo 实验表明：低层 LSTM 更偏句法信息，高层 LSTM 更偏语义信息。任务相关的加权可以让每个下游任务自行选择重要层。

2.2 预训练细节

- 数据：1B Word Benchmark 的单句
- 训练：10 个 epoch，3 张 GTX 1080 训练约 2 周
- 困惑度：前向 / 后向平均 PPL 约 39.7
- 范式：预训练只跑一次，任务无关

2.3 下游使用方式

[预训练 ELMo (冻结)] → 上下文向量 → [下游任务专属模型] → 任务输出

下游任务训练时 ELMo 参数冻结，只调整任务模型。可用于：句子分类（情感分析）、序列标注、问答、翻译.....

2.4 ELMo 的本质与局限

- 优势：预训练一次、各任务复用 contextual embeddings
- 限制：只增强词表示，下游模型架构不变——这是与下一代预训练（BERT/GPT）的根本区别

[Q] 提示

ELMo 只产出嵌入向量，能否做到「连模型权重都直接复用」？这正是 BERT/GPT 给出的答案。

3. Pre-training & Fine-tuning 范式

更现代的范式：

1. 在大规模任务 X 上预训练整个模型
2. 在下游任务 Y 上微调（fine-tune）所有权重，仅添加少量任务相关参数（如分类头）

“Fine-tuning is the process of taking the network learned by these pre-trained models, and further training the model, often via an added neural net classifier that takes the top layer of the network as input, to perform some downstream task.”

任务 X 与 Y 之间存在语义相关性，使得 X 学到的表示能力可迁移到 Y。

3.1 三类架构

Architecture	Pretraining Objective	Examples
Encoder-only	Masked Language Model (MLM)	BERT, RoBERTa, ELECTRA
Encoder-Decoder	Span Corruption	T5, BART
Decoder-only	Autoregressive Language Model	GPT-2, GPT-3, Llama

后续三章分别展开。

4. Transformer 架构核心回顾

本节预训练模型全部基于 Transformer，所以先回顾关键公式。

4.1 Self-Attention

输入向量序列 $X \in \mathbb{R}^{n \times d}$ ，通过线性投影得到 query / key / value：

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

缩放点积注意力：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

其中 $\sqrt{d_k}$ 是温度缩放，避免点积进入 softmax 饱和区导致梯度消失。

4.2 Multi-Head Attention

将 attention 并行计算 h 次，每次用不同投影矩阵：

$$\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

直觉：不同头可以关注不同位置 / 不同关系（句法、共指等）。

4.3 因果掩码 (Causal Mask)

decoder 的自注意力要求位置 t 不能看到 $t + 1, \dots, n$ ，于是在 softmax 之前加掩码 M ：

$$M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

$$\text{Attention}_{\text{causal}}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

GPT 系列使用因果掩码以维持自回归性质；BERT encoder 不使用掩码（双向）。

4.4 Encoder / Decoder Block

- Encoder block: MultiHead-Self-Attention $\rightarrow$ Add&Norm $\rightarrow$ FFN $\rightarrow$ Add&Norm
- Decoder block: Masked-Self-Attention $\rightarrow$ Add&Norm $\rightarrow$ Cross-Attention (over encoder output) $\rightarrow$ Add&Norm $\rightarrow$ FFN $\rightarrow$ Add&Norm

三种预训练架构因此可以表述为：– Encoder-only：只堆 encoder block（双向）– Decoder-only：只堆 decoder block 的「masked-self-attn + FFN」部分（单向因果）– Encoder-Decoder：完整 encoder + decoder

5. BERT：双向编码器表示

BERT: Bidirectional Encoder Representations from Transformers (Devlin et al., 2018–10)

5.1 设计要点

- 基于深层 双向 encoder-only Transformer
- 遵循 pre-training & fine-tuning 范式
- 核心目标：基于双向上下文学习表示
- 两个新的预训练目标：
 - Masked Language Modeling (MLM)
 - Next Sentence Prediction (NSP)（后续工作发现弊大于利）

5.2 Masked Language Modeling (MLM)

定义

在输入序列中随机遮蔽 $k\%$ 的 token，用模型预测被遮蔽 token：

原句：the man went to the store to buy a gallon of milk 输入：the man went to [MASK] to buy a [MASK] of milk 目标：第一个 [MASK] → “store”，第二个 [MASK] → “gallon”

BERT 取 $k = 15\%$ 。

形式化

设原句 token 为 $x = (x_1, \dots, x_n)$ ，被遮蔽位置集合为 $\mathcal{M}$ ，遮蔽后输入为 $\tilde{x}$ 。MLM 损失：

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log p_{\theta}(x_i | \tilde{x})$$

其中 $p_{\theta}(x_i | \tilde{x}) = \text{softmax}(Wh_i + b)$ ， h_i 是 encoder 在位置 i 的最终隐向量。

80 / 10 / 10 替换策略

预训练阶段会出现 [MASK] token，但下游微调时输入不含 [MASK]，造成 train-test mismatch。BERT 的缓解方案：被选中的 15% token 中：

- 80% 替换为 [MASK]：went to the store → went to the [MASK]
- 10% 替换为词表中随机词：went to the store → went to the running
- 10% 保持原样：went to the store → went to the store

这样模型必须对每个 token 的表示都保持高质量（无法判断当前位置是不是被「干扰」过）。

5.3 Next Sentence Prediction (NSP)

动机

许多下游任务需要理解两个句子之间的关系：– Natural Language Inference (NLI) – Paraphrase detection – Question Answering – Dialogue

输入格式

[CLS] Sentence A tokens ... [SEP] Sentence B tokens ... [SEP]

- [CLS]：句首特殊 token，其最终隐向量用作 NSP 分类输入
- [SEP]：句段分隔符

采样规则

- 50% 时间： B 是 A 在语料中真正的下一句 (label = IsNext)
- 50% 时间： B 是语料中随机抽的另一句 (label = NotNext)

NSP 设计动机是「缩小预训练和微调之间的差距」。但后续工作 (RoBERTa、Joshi et al. 2020、Liu et al. 2019) 指出 NSP 没用甚至有害，最终被移除。

5.4 输入表示

- 词表：30,000 WordPieces (Wu et al., 2016)
- 输入嵌入 = Token Embedding + Segment Embedding + Position Embedding (三者逐位相加)

5.5 模型规模

模型	层数 L	隐藏维度 H	注意力头数 A	参数量
BERT-base	12	768	12	110M
BERT-large	24	1024	16	340M

5.6 预训练细节

- 语料: Wikipedia (2.5B) + BooksCorpus (0.8B)
- 最大序列长度: 512 WordPiece (两段约各 256)
- 训练步数: 1M steps, 约 40 epoch, batch size 128k
- MLM 和 NSP 联合训练: [CLS] 隐向量训练 NSP, 其它 token 隐向量训练 MLM

5.7 微调

「Pre-train once, fine-tune many times.」

- 在下游数据上更新全部参数
- 不同任务接不同的轻量任务头:
 - 句子分类 → [CLS] 上加分类头
 - 序列标注 → 每个 token 上加分类头
 - 问答 (SQuAD) → 预测 answer span 的 start / end 位置
 - 句对任务 → [CLS] 上加分类头 (输入用 [SEP] 拼接)

5.8 实验结果

- GLUE: BERT-large 大幅超越当时所有方法
- SQuAD: BERT-large 在抽取式 QA 上达到 SOTA

5.9 消融实验

预训练任务

- MLM left-to-right LM: 双向上下文的关键作用
- NSP 在某些任务上有提升, 但后续工作 (Joshi 2020、Liu 2019) 认为整体无益

规模

「The bigger, the better」——增加层数 L 、隐藏维度 H 、注意力头数 A 都能稳定提升下游表现。

训练效率

- MLM 收敛比 left-to-right LM 慢, 因为 每步只预测 15% 的 token (每个 token 提供的梯度信号更少)

5.10 BERT 在学什么?

Clark et al., 2019 What Does BERT Look At? An Analysis of BERT's Attention 指出: BERT 的不同 attention 头会聚焦不同语言现象 (指代、句法依存等), 证明预训练学到了结构化语言知识。

6. RoBERTa: 把 BERT 训得更彻底

RoBERTa: A Robustly Optimized BERT Pretraining Approach (Liu et al., 2019)

核心观察: BERT 严重欠训练。RoBERTa 的改动:

1. 移除 NSP: 实验表明 NSP 引入的噪声多于收益
2. 更长训练 + 10x 数据 + 更大 batch
3. 动态掩码: 每个 epoch 重新选择遮蔽位置 (BERT 是静态预生成)
4. 2019 年在 1024 张 V100 上训练约 1 天

[NOTE] 核心启示

RoBERTa 没有改架构、没有加新目标, 只是把 BERT 的训练做扎实, 就把性能再推一截。这预示了 scaling 的重要性: **数据 + 算力 + 训练时长** 经常比新算法更管用。

7. Encoder-only 的局限

“Why not use pre-trained encoders for everything?”

If your task involves **generating sequences**, BERT and other pre-trained encoders don't naturally lead to nice **autoregressive (1-word-at-a-time) generation** methods.

也就是说, encoder-only 的 BERT 强在「读」(NLU), 但不擅长「写」(NLG)。生成任务需要 decoder。

8. T5: Text-to-Text Transfer Transformer

Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (Raffel et al., 2019)

8.1 设计哲学

“Every task ...is cast as feeding our model text as input and training it to generate some target text. This allows us to use the same model, loss function, hyperparameters, etc. across our diverse set of tasks.”

所有任务统一为 **text-to-text**: 翻译、分类、QA、摘要、回归通通转写成「输入文本 → 输出文本」。例:

translate English to German: That is good.	→ Das ist gut.
cola sentence: The course is jumping well.	→ not acceptable
sts sentence1: ... sentence2: ...	→ 3.8
summarize: <article>	→ <summary>

8.2 架构

完整 **encoder-decoder Transformer**: encoder 双向读输入, decoder 自回归生成输出。这样兼顾「双向理解」和「自回归生成」。

8.3 自监督目标: Span Corruption

T5 不做单 token MLM, 而是 **整段 (span) 掩盖**:

- 掩码比例: 15%
- 掩码单位: **span** (连续若干 token)

- 用 sentinel token (<X>、<Y>、<Z>) 替换 span
- 目标: decoder 顺序生成「sentinel + 对应内容」

举例:

原文: Thank you for inviting me to your party last week
 输入: Thank you <X> me to your party <Y> week
 目标: <X> for inviting <Y> last <Z>

形式上, 记被掩盖的 spans 集合为 $\{s_1, \dots, s_K\}$, 每个 s_k 由 sentinel z_k 替换。损失为标准的自回归交叉熵:

$$\mathcal{L}_{T5} = - \sum_{k=1}^K \log p_{\theta}(z_k, s_k \mid z_{<k}, s_{<k}, \tilde{x})$$

8.4 数据与训练

- C4 (Common Crawl): T5 团队专门做了数据清洗
- 在 NLU (GLUE/SuperGLUE) 和 NLG (CNN/DailyMail summarization、WMT 翻译) 上都拿到强势成绩

9. GPT 系列: 自回归解码器

回到三类架构表的最后一栏: decoder-only + autoregressive LM。

9.1 GPT-1

Improving Language Understanding by Generative Pre-Training, OpenAI (2018-06)

设计要点: - 用 Transformer decoder (单向、左到右) 替代 LSTM - 预训练目标: 标准语言模型 - 训练文本片段长度 512 BPE token, 而非单句

目标函数

$$\mathcal{L}_{LM}(\theta) = - \sum_{i=1}^n \log p_{\theta}(x_i \mid x_1, \dots, x_{i-1})$$

其中 p_{θ} 由带因果掩码的 Transformer decoder 给出。

下游适配

微调阶段对全部参数继续训练, 输入 / 输出格式按任务转写 (例如分类拼成「<start> 文本 <extract>」→ 在 <extract> 位置取 hidden state 接分类头)。

模型规模

项目	数值
层数	12
隐藏维度	768
注意力头数	12
参数量	110M (与 BERT-base 相当)

项目	数值
训练语料	BooksCorpus (0.8B)
最大序列	512 wordpiece
训练	100 epoch, batch size 64

GLUE 实验

GPT-1 在 GLUE 上「promising, but not exciting」——与同期 BERT 双向 + 更多数据相比稍逊。

9.2 LM vs Masked LM 的目标对比

维度	Causal LM (GPT)	Masked LM (BERT)
每次前向训练信号	每个 token 都参与预测 （密度更高）	仅 15% token 提供梯度
文本生成	天然支持（自回归）	不自然
信号质量	「更嘈杂」——只能用左侧上下文	双向上下文，质量更高
微调后表现	一般而言略逊	通常更好
收敛速度	较快	较慢

要点：LM 每个 token 都给梯度，MLM 只有 15%；但 MLM 的双向上下文让单个梯度信号更强——两条路线各有所长。

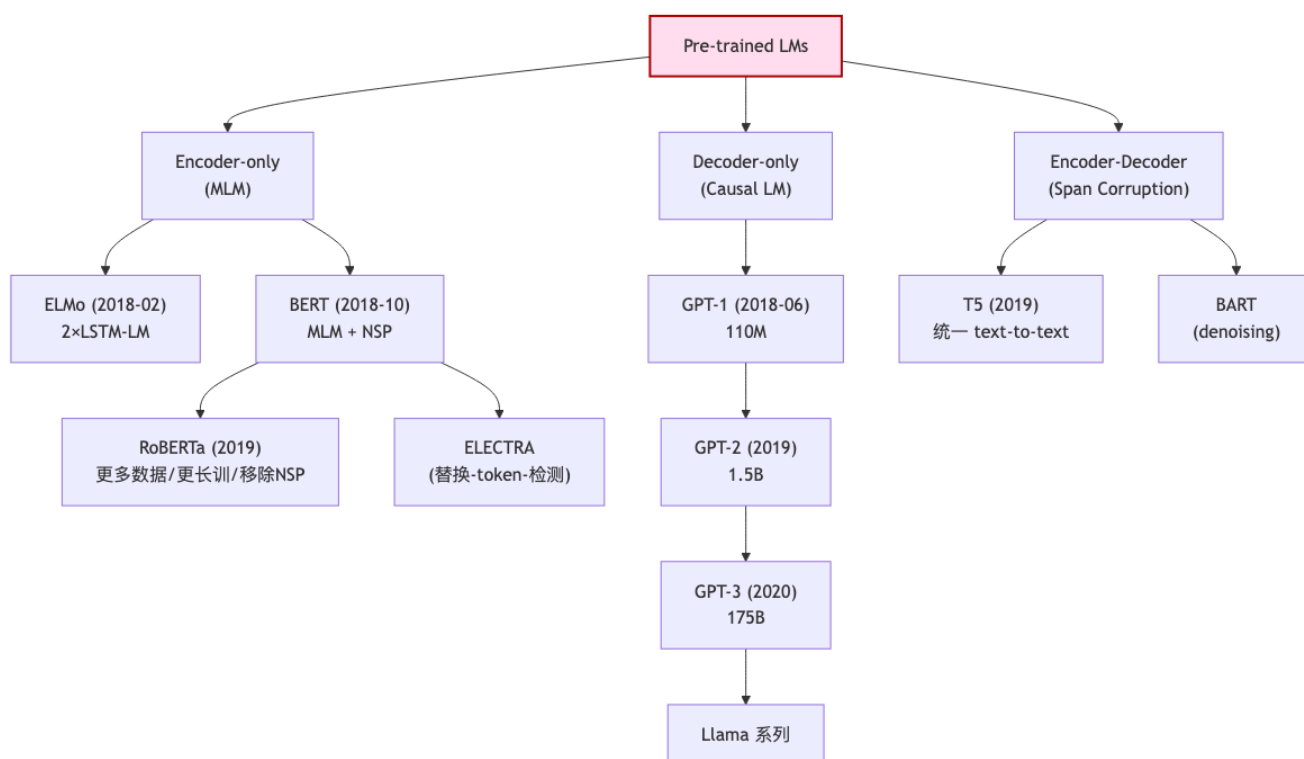
9.3 GPT-2、GPT-3: Scaling Up

模型	参数量	训练词数	发布时间
GPT-1	117 M	~1 B	2018-06
GPT-2	1.5 B	~14 B	2019-02
GPT-3	175 B	~300 B	2020-05

三件事在同步放大：– 更多参数 – 更多（更高质量）训练数据 + 更长文本 – 更多算力

到 2020 年，GPT-3 第一次大规模展示了「模型大到一定程度，不用 fine-tune 也能做新任务」的能力。

10. 模型家族 mermaid 图



注：BART / ELECTRA 仅作为家族成员列出，本课件未展开。

11. 下游适配范式

11.1 Feature Extraction (ELMo 式)

预训练模型权重冻结，只提取 contextual embedding 喂给下游任务模型。下游模型结构与预训练无关。

11.2 Fine-tuning (BERT / GPT 式，2020 之前的默认范式)

预训练模型权重全部参与下游训练，外加少量任务头。

“(Before 2020) Fine-tuning was the default.”—BERT, T5, GPT, ...

特点与代价：– 每个任务单独微调一份模型 – 任务数据量需求：SST-2 有 67k 样本，SQuAD 有 88k 三元组 – 需要计算梯度 + 全量参数更新——对大模型而言代价昂贵

11.3 范式转移的前夜

“What about the Next? Can we somehow try a new paradigm?”

GPT-3 的诞生提示了答案：**prompting / in-context learning** 不更新权重也能执行新任务，从而引出后续课程的 prompt-based learning / instruction tuning 等主题（本节不展开）。

12. 关键模型对比表

模型	年份	架构	预训练目标	参数	训练语料	是否支持生成	主要贡献
ELMo	2018-02	双向 LSTM-LM	前向 LM + 后向 LM	94 M	1B Word Benchmark	否 (仅 embedding)	上下文相关表示, 首次大规模预训练 + 任务无关
GPT-1	2018-06	Transformer decoder	Causal LM	110 M	BooksCorpus (0.8B)	是	首次用 Transformer 做生成式预训练 + 全权重微调
BERT-base	2018-10	Transformer encoder	MLM + NSP	110 M	Wiki + Books (3.3B)	否 (生成不自然)	双向 MLM, 预训练-微调成为通用范式
BERT-large	2018-10	Transformer encoder	MLM + NSP	340 M	同上	否	更大模型, GLUE / SQuAD SOTA
GPT-2	2019	Transformer decoder	Causal LM	1.5 B	~14 B 词	是	显示 scaling 的潜力, zero-shot 初见
RoBERTa	2019	Transformer encoder	MLM (无 NSP)	355 M	160 GB (10x BERT)	否	证明 BERT 被严重欠训练
T5	2019	Encoder-Decoder	Span Corruption	11 B (最大)	C4 (清洗后的 Common Crawl)	是	统一 text-to-text 框架
GPT-3	2020	Transformer decoder	Causal LM	175 B	~300 B 词	是	Few-shot prompting; 推动范式转向 in-context learning

13. 期末复习要点

13.1 概念辨析

1. 静态词向量 vs 上下文向量: word2vec 一词一向量; ELMo / BERT 同词在不同语境下向量不同
2. ELMo 范式 vs BERT/GPT 范式: ELMo 冻结预训练模型, 只把 embedding 注入; BERT/GPT 整体微调
3. 三类架构: encoder-only / decoder-only / encoder-decoder 对应 MLM / Causal LM / Span Corruption
4. MLM 80/10/10 替换: 缓解 [MASK] 在下游不出现的 train-test 不匹配
5. NSP 命运: BERT 引入 → RoBERTa 移除 → 现今被普遍认为弊大于利

13.2 必背公式

- 缩放点积注意力: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$
- MLM 损失: $\mathcal{L}_{\text{MLM}} = -\sum_{i \in \mathcal{M}} \log p_\theta(x_i | \tilde{x})$
- Causal LM 损失: $\mathcal{L}_{\text{LM}} = -\sum_{i=1}^n \log p_\theta(x_i | x_{<i})$
- ELMo 加权: $\text{ELMo}_k^{\text{task}} = \gamma^{\text{task}} \sum_j s_j^{\text{task}} h_{k,j}$

13.3 数字记忆点

事项	数值
BERT-base 参数 / 层 / 头 / 隐藏	110M / 12 / 12 / 768
BERT-large 参数 / 层 / 头 / 隐藏	340M / 24 / 16 / 1024
BERT 词表	30,000 WordPieces
BERT 训练语料	Wiki 2.5B + Books 0.8B
BERT 最大序列	512
MLM 掩码比例	15% (80 / 10 / 10 拆分)
T5 span 掩码比例	15%
GPT-1 / 2 / 3 参数	117M / 1.5B / 175B
GPT-3 训练词数	~300B

13.4 高频追问

- 为何 MLM 比 left-to-right LM 在 NLU 任务上更强? 双向上下文使每个位置都能用左右两侧信息, 对理解任务 (分类、抽取式 QA) 更有利
- 为何 GPT 仍然选择 Causal LM? 自回归性质天然支持文本生成; 同时每个 token 都贡献梯度, 训练信号密度更高
- 为何 ELMo 用前后向独立 LM 而不是真双向? 真双向 LM 会让目标 token 在条件中「泄露」自己, 无法形成有效语言模型目标。BERT 用 MLM 解决了这个问题
- 为何 ELMo 加权多层而非取顶层? 不同层捕获不同语言信息 (低层句法、高层语义), 任务相关加权比硬选某层更灵活
- 为何 BERT 训练时 MLM 收敛慢? 每步只对 15% 的 token 计算 loss, 单次前向获得的梯度信号少
- RoBERTa 启示是什么? 训练时长、数据规模、batch size、动态掩码这些「工程细节」对最终性能极为关键; 不要急于发明新目标
- 2020 前后预训练范式发生了什么转移? 从「每任务都 fine-tune」转向「一次预训练 + prompt / in-context learning 处理多任务」, 由 GPT-3 推动